

جامعة نيويورك أبوظبي



Introduction to Artificial Intelligence

Uninformed Search I

Julian Barreiro-Gomez

2026

Uninformed Search Strategies

- ▶ Also known as **blind search**
- ▶ The strategies have no additional information about **states** beyond **problem definition**
- ▶ Generate **successors** and **distinguish** a goal state
- ▶ **Order** in which nodes are **expanded**

Non-goal state is "more promising"?
This is called **informed search** or **heuristic search**

Uninformed Search Strategies

The main *Uninformed Search Strategies* are

- ▶ Breadth-first search (**BFS**)
- ▶ Uniform-cost search (**UCS**)
- ▶ Depth-first search (**DFS**)
- ▶ Depth-limited search (**DLS**)
- ▶ Iterative deepening depth-first search (**IDDFS**)
- ▶ Bidirectional search (**BS**)

Breadth-first search (BFS)

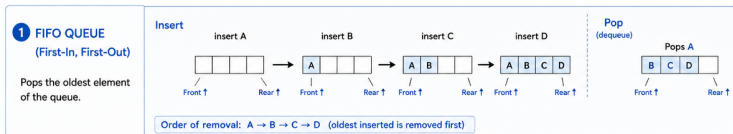
Breadth-first search (BFS)

Main Idea of Breadth-First Search

Breadth-First Search (BFS) explores the search space level by level.

- ▶ First, it explores all nodes at depth 0.
- ▶ Then all nodes at depth 1.
- ▶ Then all nodes at a depth 2.
- ▶ And so on.

To achieve this behavior, BFS uses a **FIFO (First-In First-Out)** queue.



Breadth-first search (BFS)

```
1: function BREADTH-FIRST-SEARCH(problem)
2:   node  $\leftarrow$  a node with
       STATE = problem.INITIAL-STATE
       PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier  $\leftarrow$  a FIFO queue with node as the only element
7:   explored  $\leftarrow \emptyset$ 
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:    end if
12:    node  $\leftarrow$  POP(frontier)           ▷ Chooses the shallowest node
13:    explored  $\leftarrow$  explored  $\cup$  {node.STATE}
14:    for all action  $\in$  problem.ACTIONS(node.STATE) do
15:      child  $\leftarrow$  CHILD-NODE(problem,node,action)
16:      if child.STATE  $\notin$  explored and child.STATE  $\notin$  frontier then
17:        if problem.GOAL-TEST(child.STATE) then
18:          return SOLUTION(child)
19:        end if
20:        INSERT(child, frontier)
21:      end if
22:    end for
23:  end while
24: end function
```

Data Structures

Frontier. The frontier contains nodes that have been discovered but not yet expanded.

Example: Frontier = $[A, B, C]$. If we perform

POP(frontier),

the node A is removed first because the frontier is a FIFO queue:

$[A, B, C] \rightarrow [B, C]$.

Explored Set. The explored set stores states that have already been expanded.

Example: Explored = $\{A, B\}$. This prevents revisiting the same state and avoids infinite loops in cyclic graphs.

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
       STATE = problem.INITIAL-STATE
       PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← {}
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 2

```

node ← initial state
path-cost ← 0

```

- Create **root node** of search tree
- If the **initial state** is A , then

$node = A.$

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
         STATE = problem.INITIAL-STATE
         PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← {}
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 2

```

node ← initial state
path-cost ← 0

```

- ▶ Create **root node** of search tree
- ▶ If the **initial state** is A , then

$node = A$.

Line 3–5

```

if GOAL-TEST(node.STATE)
  return solution

```

- ▶ **Already goal?**
- ▶ If yes, **Algorithm Terminates**

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← {}
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 6

```

frontier <- \\
FIFO queue containing node

```

► Initialize frontier:

Frontier = [A].

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
         STATE = problem.INITIAL-STATE
         PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 6

frontier ← \\
FIFO queue containing node

► Initialize frontier:

Frontier = [A].

Line 7

explored ← empty set

► Initially no state explored:

Explored = ∅.

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node  $\leftarrow$  a node with
       STATE = problem.INITIAL-STATE
       PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier  $\leftarrow$  a FIFO queue with node as the only element
7:   explored  $\leftarrow \emptyset$ 
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node  $\leftarrow$  POP(frontier)           ▷ Chooses the shallowest node
13:     explored  $\leftarrow$  explored  $\cup$  {node.STATE}
14:     for all action  $\in$  problem.ACTIONS(node.STATE) do
15:       child  $\leftarrow$  CHILD-NODE(problem,node,action)
16:       if child.STATE  $\notin$  explored and child.STATE  $\notin$  frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 8

while true

Main search loop.

Algorithm continues until:

- ▶ Solution found, or
- ▶ Frontier empty

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:    end if
12:    node ← POP(frontier)           ▷ Chooses the shallowest node
13:    explored ← explored ∪ {node.STATE}
14:    for all action ∈ problem.ACTIONS(node.STATE) do
15:      child ← CHILD-NODE(problem,node,action)
16:      if child.STATE ∉ explored and child.STATE ∉ frontier then
17:        if problem.GOAL-TEST(child.STATE) then
18:          return SOLUTION(child)
19:        end if
20:        INSERT(child, frontier)
21:      end if
22:    end for
23:  end while
24: end function
    
```

Line 8

while true
Main search loop.

Algorithm continues until:

- ▶ Solution found, or
- ▶ Frontier empty

Line 9–11

if EMPTY(frontier)
return failure

- ▶ If no nodes to examine, then no solution exists.

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:    end if
12:    node ← POP(frontier)           ▷ Chooses the shallowest node
13:    explored ← explored ∪ {node.STATE}
14:    for all action ∈ problem.ACTIONS(node.STATE) do
15:      child ← CHILD-NODE(problem,node,action)
16:      if child.STATE ∉ explored and child.STATE ∉ frontier then
17:        if problem.GOAL-TEST(child.STATE) then
18:          return SOLUTION(child)
19:        end if
20:        INSERT(child, frontier)
21:      end if
22:    end for
23:  end while
24: end function
    
```

Line 12

node ← POP(frontier)

- ▶ Remove **first node** from **FIFO queue**
- ▶ **FIFO**: shallow nodes are expanded before deeper nodes

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ≠ explored and child.STATE ≠ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function
    
```

Line 12

node ← POP(frontier)

- ▶ Remove **first node** from **FIFO queue**
- ▶ **FIFO: shallow nodes are expanded before deeper nodes**

Line 13

add node.STATE to explored

- ▶ Mark **current state** as **expanded**
- ▶ For example,

Explored = {A}.

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
       STATE = problem.INITIAL-STATE
       PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ≠ explored and child.STATE ≠ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function
  
```

Line 14

for each action in \\
ACTIONS(node.STATE)

- Generate **all possible actions** available from current state.

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
         STATE = problem.INITIAL-STATE
         PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 14

for each action in \\
ACTIONS(node.STATE)

- Generate **all possible actions** available from current state.

Line 15

child ← CHILD-NODE(...)

- Create **successor node** corresponding to chosen action

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function

```

Line 16

if child not in explored \\
and not in frontier

- ▶ Avoid duplicate states
- ▶ Only unexplored and undiscovered states are inserted

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
           STATE = problem.INITIAL-STATE
           PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:     end if
12:     node ← POP(frontier)           ▷ Chooses the shallowest node
13:     explored ← explored ∪ {node.STATE}
14:     for all action ∈ problem.ACTIONS(node.STATE) do
15:       child ← CHILD-NODE(problem,node,action)
16:       if child.STATE ∉ explored and child.STATE ∉ frontier then
17:         if problem.GOAL-TEST(child.STATE) then
18:           return SOLUTION(child)
19:         end if
20:         INSERT(child, frontier)
21:       end if
22:     end for
23:   end while
24: end function
    
```

Line 16

if child not in explored \\
and not in frontier

- ▶ Avoid duplicate states
- ▶ Only unexplored and undiscovered states are inserted

Line 17–18

if GOAL-TEST(child.STATE)
return solution

- ▶ Generated child is the goal?
- ▶ If yes, terminate

Breadth-first search (BFS)

```

1: function BREADTH-FIRST-SEARCH(problem)
2:   node ← a node with
        STATE = problem.INITIAL-STATE
        PATH-COST = 0
3:   if problem.GOAL-TEST(node.STATE) then
4:     return SOLUTION(node)
5:   end if
6:   frontier ← a FIFO queue with node as the only element
7:   explored ← ∅
8:   while true do
9:     if EMPTY(frontier) then
10:      return failure
11:    end if
12:    node ← POP(frontier)           ▷ Chooses the shallowest node
13:    explored ← explored ∪ {node.STATE}
14:    for all action ∈ problem.ACTIONS(node.STATE) do
15:      child ← CHILD-NODE(problem,node,action)
16:      if child.STATE ∉ explored and child.STATE ∉ frontier then
17:        if problem.GOAL-TEST(child.STATE) then
18:          return SOLUTION(child)
19:        end if
20:        INSERT(child, frontier)
21:      end if
22:    end for
23:  end while
24: end function
    
```

Line 19

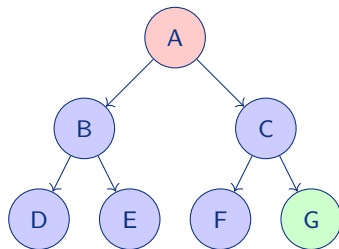
INSERT(child, frontier)

- ▶ Insert **child** at the end of queue
- ▶ This preserves **FIFO** ordering

Breadth-first search (BFS) - Example

Breadth-first search (BFS) - Toy Example

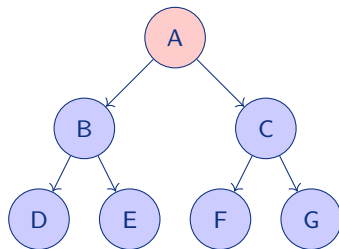
Consider the search tree



Initial state: A
Goal state: G

Breadth-first search (BFS) - Toy Example

Consider the search tree



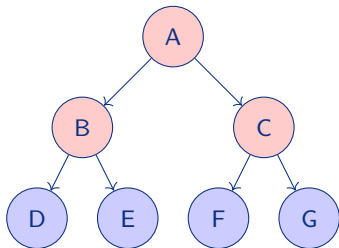
Initial state: A
Goal state: G

Initialization

Frontier = [A]
Explored = $\emptyset$.

Breadth-first search (BFS) - Toy Example

Consider the search tree



Initial state: A
Goal state: G

Initialization

Frontier = [A]
Explored = $\emptyset$.

Iteration 1

Pop first node:

node = A.

Add A to **explored**:

Explored = {A}.

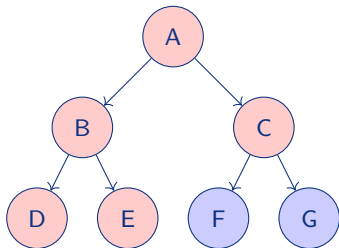
Generate children: B, C.

Insert them:

Frontier = [B, C].

Breadth-first search (BFS) - Toy Example

Consider the search tree



Initial state: A
Goal state: G

Iteration 2

Pop: node = B .

Frontier becomes: $[C]$.

Add B to **explored**:

$\text{Explored} = \{A, B\}$.

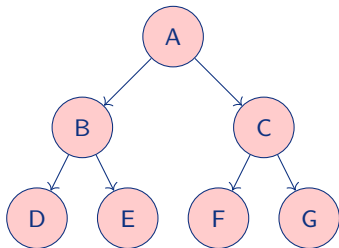
Generate children: D, E .

Insert them:

$\text{Frontier} = [C, D, E]$.

Breadth-first search (BFS) - Toy Example

Consider the search tree



Initial state: A
Goal state: G

Iteration 3

Pop: node = C.

Frontier becomes:

$[D, E]$.

Add C to **explored**:

$\text{Explored} = \{A, B, C\}$.

Generate children:

F, G .

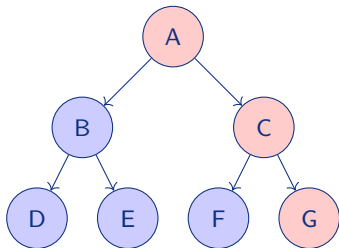
Insert them:

$\text{Frontier} = [D, E, F, G]$.

Pop: node = G. G is goal, algorithm stops.

Breadth-first search (BFS) - Toy Example

Consider the search tree



Initial state: A
Goal state: G

Recovering the Solution

Therefore, BFS returns

$A \rightarrow C \rightarrow G$

BFS expands nodes by increasing depth:

- ▶ Depth 0 : A
- ▶ Depth 1 : B, C
- ▶ Depth 2 : D, E, F, G.

The expansion order is

A, B, C, D, E, F, G.

Thank you